

PACMANN for convection dominant flows

1 Methodology

This section briefly reviews the framework of physics-informed neural networks (PINNs) and adapts it to the convection–diffusion equation.

1.1 Physics-informed neural networks for the convection–diffusion equation

We consider the time-dependent convection–diffusion equation defined on a spatial domain $\Omega \subset \mathbb{R}^d$ and time interval $[0, T]$:

$$u_t + \boldsymbol{\beta} \cdot \nabla u - \epsilon \Delta u = f(x, t), \quad x \in \Omega, \quad t \in [0, T], \quad (1.1)$$

where $\boldsymbol{\beta}$ denotes the convection velocity field, $\epsilon > 0$ is the diffusion coefficient, and $f(x, t)$ represents a source term. The problem is supplemented with the initial condition

$$u(x, 0) = h(x), \quad x \in \Omega, \quad (1.2)$$

and boundary conditions

$$u(x, t) = g(x, t), \quad x \in \partial\Omega, \quad t \in [0, T]. \quad (1.3)$$

Let $\hat{u}(x, t, \theta)$ denote the neural network approximation of the solution $u(x, t)$, where θ represents the trainable parameters of the neural network. The network takes (x, t) as input and outputs an approximation of the solution. The parameters θ are obtained by minimizing the loss function

$$\theta^* = \arg \min_{\theta} \mathcal{L}(\theta). \quad (1.4)$$

The total loss function consists of three components:

$$\mathcal{L}(\theta) = \lambda_r \mathcal{L}_r(\theta) + \lambda_{ic} \mathcal{L}_{ic}(\theta) + \lambda_{bc} \mathcal{L}_{bc}(\theta), \quad (1.5)$$

where λ_r , λ_{ic} , and λ_{bc} are weighting parameters.

PDE residual loss

The residual of the convection–diffusion equation is defined as

$$r(x, t, \theta) = \hat{u}_t + \boldsymbol{\beta} \cdot \nabla \hat{u} - \epsilon \Delta \hat{u} - f(x, t). \quad (1.6)$$

The derivatives $\hat{u}_t$, $\nabla \hat{u}$, and $\Delta \hat{u}$ appearing in the above expression are computed via automatic differentiation. This allows the PDE residual to be evaluated accurately at the interior collocation

points without explicitly discretizing the differential operators. The PDE residual loss is evaluated at N_r collocation points (x_r^i, t_r^i) inside the domain:

$$\mathcal{L}_r(\theta) = \frac{1}{N_r} \sum_{i=1}^{N_r} r(x_r^i, t_r^i, \theta)^2. \quad (1.7)$$

Initial condition loss

The loss corresponding to the initial condition is defined as

$$\mathcal{L}_{ic}(\theta) = \frac{1}{N_{ic}} \sum_{i=1}^{N_{ic}} (\hat{u}(x_{ic}^i, 0, \theta) - h(x_{ic}^i))^2. \quad (1.8)$$

Boundary condition loss

The boundary condition loss is given by

$$\mathcal{L}_{bc}(\theta) = \frac{1}{N_{bc}} \sum_{i=1}^{N_{bc}} (\hat{u}(x_{bc}^i, t_{bc}^i, \theta) - g(x_{bc}^i, t_{bc}^i))^2. \quad (1.9)$$

Here, $\{(x_r^i, t_r^i)\}_{i=1}^{N_r}$ denote collocation points in the interior of the space-time domain, $\{(x_{ic}^i, 0)\}_{i=1}^{N_{ic}}$ denote points sampled at the initial time, and $\{(x_{bc}^i, t_{bc}^i)\}_{i=1}^{N_{bc}}$ represent points sampled along the boundary $\partial\Omega$. These points may be fixed during training, randomly resampled, or adaptively redistributed based on error indicators such as the PDE residual.

To train the model parameters θ , the gradient of the loss function with respect to the parameters is computed using the backpropagation algorithm. The parameters are then updated using an optimization scheme based on gradient descent, such as the Adam optimizer.

For problems that incorporate reference data during training, an additional loss term $\mathcal{L}_{ref}$ is added to the loss function. This term penalizes the discrepancy between observed data and the neural network prediction. It is defined as

$$\mathcal{L}_{ref}(\theta) = \frac{1}{N_{ref}} \sum_{i=1}^{N_{ref}} (\hat{u}(x^i, t^i, \theta) - u_{ref}(x^i, t^i))^2, \quad (1.10)$$

where $u_{ref}(x^i, t^i)$ denotes the observed (possibly noisy) reference data at the data points $\{(x^i, t^i)\}_{i=1}^{N_{ref}}$. This term represents the mean squared error between the neural network approximation $\hat{u}(x^i, t^i, \theta)$ and the observed data. Incorporating this additional constraint improves the robustness of the solution when measurement data are available.

The resulting loss function used to train the neural network becomes

$$\mathcal{L}(\theta) = \lambda_r \mathcal{L}_r(\theta) + \lambda_{ic} \mathcal{L}_{ic}(\theta) + \lambda_{bc} \mathcal{L}_{bc}(\theta) + \lambda_{ref} \mathcal{L}_{ref}(\theta), \quad (1.11)$$

where λ_r , λ_{ic} , λ_{bc} , and λ_{ref} are weighting parameters used to balance the contributions of the PDE residual, the initial condition, the boundary condition, and the reference data loss terms, respectively.

1.2 Point adaptive collocation method for artificial neural networks (PACMANN)

In this work, we employ the Point Adaptive Collocation Method for Artificial Neural Networks (PACMANN) to improve the training of physics-informed neural networks for the convection–diffusion equation. The method utilizes the gradient of the squared residual of the governing PDE to adaptively move the collocation points toward regions where the residual is large.

Instead of solving the standard minimization problem in Eq. (1.11), we consider the following min–max optimization problem for the model parameters θ :

$$\theta^* = \arg \min_{\theta} \left[\lambda_{ic} \mathcal{L}_{ic}(\theta) + \lambda_{bc} \mathcal{L}_{bc}(\theta) + \lambda_{ref} \mathcal{L}_{ref}(\theta) + \lambda_r \max_{X_r \subset D} \mathcal{L}_r(X_r, \theta) \right], \quad (1.12)$$

where $X_r = \{(x_r^i, t_r^i)\}_{i=1}^{N_r}$ denotes the set of interior collocation points.

During the PACMANN procedure, only the collocation points X_r are moved according to the gradient of the squared residual. The points located on the boundary and those associated with the initial condition remain fixed throughout the training process in order to accurately enforce the corresponding constraints.

Remark. The min–max formulation in Eq. (1.12) can be interpreted as minimizing the supremum norm of the residual of the convection–diffusion equation over the space–time domain D . In particular,

$$\|r(x, t)\|_{L^\infty(D)}^2 = \max_{(x, t) \in D} (\hat{u}_t + \beta \cdot \nabla \hat{u} - \varepsilon \Delta \hat{u} - f)^2 \quad (1.13)$$

$$= \max_{X_r \subset D} \left[\frac{1}{N_r} \sum_{i=1}^{N_r} r(x_r^i, t_r^i, \theta)^2 \right] \quad (1.14)$$

$$= \max_{X_r \subset D} \mathcal{L}_r(X_r, \theta). \quad (1.15)$$

Therefore, the PACMANN strategy focuses the collocation points in regions where the residual of the convection–diffusion equation is largest.

In the PACMANN algorithm, the collocation points are adaptively moved toward regions where the squared residual is large. Initially, N_r collocation points are sampled in the space–time domain D using a uniform sampling strategy, such as an equispaced grid or a low–discrepancy sequence. The PINN is then trained for P iterations using these collocation points. More precisely, we evaluate

$$\nabla_x r^2(x, t, \theta), \quad \frac{\partial}{\partial t} r^2(x, t, \theta),$$

which are obtained efficiently by automatic differentiation.

The collocation points are then moved in the direction of increasing residual according to

$$\begin{aligned} x_r^{i+1} &= x_r^i + s \nabla_x r^2(x_r^i, t_r^i, \theta), \\ t_r^{i+1} &= t_r^i + s \frac{\partial}{\partial t} r^2(x_r^i, t_r^i, \theta), \end{aligned} \quad (1.16)$$

where $s > 0$ is a stepsize parameter controlling the magnitude of each update.

During the point–movement stage, the neural network parameters θ are kept fixed. Hence, the residual landscape remains unchanged while the collocation points are moved. The residual

gradients can therefore be recomputed at the new point locations and the movement process can be repeated several times. The number of such updates is controlled by the hyperparameter T , referred to as the number of point-movement steps. If a collocation point leaves the space-time domain D , it is removed from the set X_r and replaced by a new point sampled uniformly from D . After the movement phase is completed, the neural network is again trained for another P iterations and the entire procedure is repeated until a stopping criterion is reached.

In Eq. (1.16), the interior collocation points are moved by a gradient-ascent type update based on the squared residual. More generally, let x denote a variable and let $f(x)$ be the objective function to be maximized. The following optimization algorithms may be used for the PACMANN point update.

1. Gradient ascent:

$$x_{i+1} = x_i + sf'(x_i).$$

2. Nonlinear gradient ascent:

$$x_{i+1} = x_i + s \tanh(f'(x_i)).$$

3. RMSprop:

$$S_{i+1} = \beta S_i + (1 - \beta)(f'(x_i))^2,$$

$$x_{i+1} = x_i + s \frac{f'(x_i)}{\sqrt{S_{i+1} + \varepsilon}}.$$

4. Momentum:

$$V_{i+1} = \beta V_i + (1 - \beta)f'(x_i),$$

$$x_{i+1} = x_i + sV_{i+1}.$$

5. Adam:

$$V_{i+1} = \beta_1 V_i + (1 - \beta_1)f'(x_i),$$

$$S_{i+1} = \beta_2 S_i + (1 - \beta_2)(f'(x_i))^2,$$

$$\hat{V}_{i+1} = \frac{V_{i+1}}{1 - \beta_1^{i+1}}, \quad \hat{S}_{i+1} = \frac{S_{i+1}}{1 - \beta_2^{i+1}},$$

$$x_{i+1} = x_i + s \frac{\hat{V}_{i+1}}{\sqrt{\hat{S}_{i+1} + \varepsilon}}.$$

6. Golden section search:

$$b_0 = a_0 + sf'(x_0),$$

$$x_{l,i} = a_i + \alpha(b_i - a_i), \quad x_{r,i} = a_i + \beta(b_i - a_i),$$

$$\phi = \frac{1 + \sqrt{5}}{2}, \quad \alpha = 1 - \phi^{-1}, \quad \beta = \phi^{-1}.$$

If $f(x_{l,i}) > f(x_{r,i})$, then

$$a_{i+1} = a_i, \quad b_{i+1} = x_{r,i}, \quad x_{r,i+1} = x_{l,i}.$$

If $f(x_{l,i}) < f(x_{r,i})$, then

$$a_{i+1} = x_{l,i}, \quad b_{i+1} = b_i, \quad x_{l,i+1} = x_{r,i}.$$

After the final iteration,

$$x_N = \frac{a_N + b_N}{2}.$$

In the present work, the function $f(x)$ corresponds to the squared residual of the convection–diffusion equation, and the optimizer is applied to relocate the interior collocation points toward regions where the residual is large.

Example

$$\begin{cases} u_t + \beta u_x - \varepsilon u_{xx} = f(x, t), & (x, t) \in [-1, 1] \times [0, 1], \\ u(x, 0) = \sin(\pi x), & x \in [-1, 1], \\ u(-1, t) = u(1, t) = 0, & t \in [0, 1]. \end{cases} \quad (1.17)$$

The exact solution is chosen as

$$u(x, t) = e^{-t} \sin(\pi x). \quad (1.18)$$

The corresponding source term is

$$f(x, t) = -e^{-t} \sin(\pi x) + \beta \pi e^{-t} \cos(\pi x) + \varepsilon \pi^2 e^{-t} \sin(\pi x). \quad (1.19)$$

Example

$$\begin{cases} u_t + \beta u_x - \varepsilon u_{xx} = f(x, t), & (x, t) \in [0, 1] \times [0, 1], \\ u(x, 0) = x - \frac{e^{(x-1)/\varepsilon} - e^{-1/\varepsilon}}{1 - e^{-1/\varepsilon}}, & x \in [0, 1], \\ u(0, t) = u(1, t) = 0, & t \in [0, 1]. \end{cases} \quad (1.20)$$

The exact solution is chosen as

$$u(x, t) = e^{-t} \left(x - \frac{e^{(x-1)/\varepsilon} - e^{-1/\varepsilon}}{1 - e^{-1/\varepsilon}} \right). \quad (1.21)$$

The corresponding source term is

$$f(x, t) = e^{-t} (-q(x) + \beta q'(x) - \varepsilon q''(x)), \quad (1.22)$$

where

$$q(x) = x - \frac{e^{(x-1)/\varepsilon} - e^{-1/\varepsilon}}{1 - e^{-1/\varepsilon}}, \quad (1.23)$$

with

$$q'(x) = 1 - \frac{e^{(x-1)/\varepsilon}}{\varepsilon (1 - e^{-1/\varepsilon})}, \quad (1.24)$$

and

$$q''(x) = -\frac{e^{(x-1)/\varepsilon}}{\varepsilon^2 (1 - e^{-1/\varepsilon})}. \quad (1.25)$$

Hence, the source term may be written explicitly as

$$f(x, t) = e^{-t} \left[-\left(x - \frac{e^{(x-1)/\varepsilon} - e^{-1/\varepsilon}}{1 - e^{-1/\varepsilon}} \right) + \beta \left(1 - \frac{e^{(x-1)/\varepsilon}}{\varepsilon (1 - e^{-1/\varepsilon})} \right) + \frac{e^{(x-1)/\varepsilon}}{\varepsilon (1 - e^{-1/\varepsilon})} \right]. \quad (1.26)$$